

Лабораторная работа №1

Julia. Установка и настройка. Основные принципы

Умед Джамshedович Назармамадов

В отчёте описана подготовка рабочего окружения для работы с языком программирования Julia: установка Julia, Jupyter Lab и ядра IJulia в среде WSL Ubuntu.

Воспроизведены базовые примеры синтаксиса Julia (типы данных, преобразование типов, функции, векторы и матрицы) и выполнены четыре задания для самостоятельной работы: функции ввода-вывода, функция `parse()`, арифметика с разными типами данных, операции над матрицами и векторами.

Все результаты подтверждены скриншотами выполнения кода в Jupyter Lab.

Содержание

1	Введение	7
1.1	Актуальность	7
1.2	Цель работы	7
1.3	Задачи	7
1.4	Объект и предмет исследования	8
1.5	Методы исследования	8
1.6	Схема выполнения работы	8
2	Теоретическая часть	9
3	Материалы и методы	10
4	Выполнение лабораторной работы	11
4.1	Подготовка инструментария к работе	11
4.2	Основы синтаксиса Julia (примеры раздела 1.3.3)	13
4.3	Задания для самостоятельной работы (раздел 1.3.4)	18
5	Результаты	27
6	Обсуждение результатов	28
7	Заключение	29
	Список литературы	30
A	Листинг кода блокнота intro-julia.ipynb	31

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
4.1	Проверка версий Julia и Jupyter	11
4.2	Установка пакета IJulia через Pkg	12
4.3	Установка Jupyter Lab	12
4.4	Создание каталога labs/lab01/project	12
4.5	Запуск Jupyter Lab	13
4.6	Определение типа числовых величин	13
4.7	Специальные значения Inf, -Inf, NaN	14
4.8	Тип специальных значений	14
4.9	Диапазоны целочисленных типов	14
4.10	Преобразование типов конструкторами	14
4.11	Преобразование типов через convert()	15
4.12	Преобразование в Bool	15
4.13	Приведение аргументов к общему типу	15
4.14	Тип результата promote()	15
4.15	Функция в полной форме	16
4.16	Функция в краткой форме	16
4.17	Векторы и обращение к элементам	17
4.18	Матрица и обращение к элементам	17
4.19	Операция с транспонированием вектора	17
4.20	print, println и show	18
4.21	write() в буфер	19
4.22	read(), readline(), readlines()	20
4.23	readdlm() для табличных данных	21
4.24	parse() для базовых типов	21
4.25	parse() с указанием системы счисления	21
4.26	tryparse()	22
4.27	Арифметика со смешанными типами	22
4.28	Операторы деления и остатка	22
4.29	Степени и корни	23
4.30	Сравнение значений и типов	23
4.31	Логические операции	23
4.32	Операции с комплексными числами	24
4.33	Операции над векторами	24
4.34	Скалярное произведение векторов	25

4.35	Сложение и вычитание матриц	25
4.36	Транспонирование матрицы	25
4.37	Умножение матриц: на скаляр, матричное и поэлементное	26

Список таблиц

1 Введение

1.1 Актуальность

Julia — язык программирования, ориентированный на научные и численные вычисления, сочетающий динамическую типизацию с производительностью, близкой к компилируемым языкам за счёт JIT-компиляции через LLVM. Он используется в курсе «Компьютерный практикум по статистическому анализу данных» как основной инструмент для последующих лабораторных работ, поэтому важно на первом занятии освоить установку окружения и базовый синтаксис.

1.2 Цель работы

Подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомиться с основами синтаксиса Julia.

1.3 Задачи

1. Установить Julia и Jupyter под свою операционную систему.
2. Используя Jupyter Lab, повторить примеры из раздела 1.3.3 методических указаний.
3. Выполнить задания для самостоятельной работы (раздел 1.3.4): функции ввода-вывода, `parse()`, арифметика с разными типами, операции над

матрицами и векторами.

1.4 Объект и предмет исследования

Объект исследования — язык программирования Julia и среда интерактивной разработки Jupyter Lab. Предмет исследования — базовые конструкции синтаксиса Julia: система типов, преобразование и приведение типов, функции, работа с векторами и матрицами.

1.5 Методы исследования

Практическая работа в интерактивной среде Jupyter Lab (ядро Julia): последовательное выполнение примеров кода в ячейках блокнота с анализом полученных результатов.

1.6 Схема выполнения работы

На рис. -Рисунок 1.1 представлена общая последовательность действий при выполнении работы.



Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Julia использует динамическую типизацию с системой параметрических типов (например, `Int64`, `Float64`, `Complex{Float64}`), между которыми при смешанных операциях действует автоматическое повышение типа (*type promotion*) — более узкий тип приводится к более широкому. Для явного управления типами служат конструкторы типов (`Int64(x)`), функция `convert()` и функция `promote()`, приводящая набор аргументов к общему типу.

Интерактивная работа с Julia в этой лабораторной ведётся через Jupyter — пакет `IJulia` регистрирует ядро Julia в Jupyter, после чего блокноты `.ipynb` можно выполнять как в Jupyter Notebook, так и в Jupyter Lab.

3 Материалы и методы

Работа выполнена в следующем окружении:

- ОС: WSL2 Ubuntu (Windows);
- Julia 1.10.5;
- Jupyter Lab 4.6.3, ядро IJulia 1.34.4;
- рабочее пространство организовано по шаблону курса (`course-directory-student-t` лабораторная — в ветке `feature/lab01` git-репозитория `2026-2--study--computer`

Последовательность действий: проверка и установка недостающих компонентов (Julia уже был установлен ранее, дополнительно установлены IJulia и jupyterlab), запуск Jupyter Lab, создание блокнота `intro-julia.ipynb` с ядром Julia 1.10, последовательное выполнение примеров и заданий с фиксацией результата скриншотом.

4 Выполнение лабораторной работы

4.1 Подготовка инструментария к работе

Проверены версии уже установленных компонентов (рис. -Рисунок 4.1).

```
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ julia --v
ersion
julia version 1.10.5
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ jupyter -
-version
Selected Jupyter core packages...
IPython          : 8.20.0
ipykernel        : 6.29.3
ipywidgets       : 8.1.1
jupyter_client   : 7.4.9
jupyter_core     : 5.3.2
jupyter_server   : not installed
jupyterlab       : not installed
nbclient         : 0.8.0
nbconvert        : 6.5.3
qtconsole        : not installed
traitlets        : 5.5.0
```

Рисунок 4.1: Проверка версий Julia и Jupyter

Установлен пакет `IJulia`, добавляющий ядро Julia в Jupyter (рис. -Рисунок 4.2).

```

umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ julia -e
'import Pkg; Pkg.add("IJulia")'
Updating registry at '~/.julia/registries/General.toml'
Resolving package versions...
Installed Parsers - v3.0.0
Installed JSON - v1.8.0
Updating ~/.julia/environments/v1.10/Project.toml
[7073ff75] + IJulia v1.34.4
Updating ~/.julia/environments/v1.10/Manifest.toml
[8f4d0f93] + Conda v1.10.3
[7073ff75] + IJulia v1.34.4
[682c06a0] + JSON v1.8.0
[69de0a69] + Parsers v3.0.0
[ec057cc2] + StructUtils v2.8.5
[81def892] + VersionParsing v1.3.0
[c2297ded] + ZMQ v1.5.1
[8f1865be] + ZeroMQ_jll v4.3.6+0
[a9144af2] + libsodium_jll v1.0.21+0
Precompiling project...
216 dependencies successfully precompiled in 351 seconds. 104 already precompiled.
2 dependencies had output during precompilation:
└─ ZMQ

```

Рисунок 4.2: Установка пакета IJulia через Pkg

Jupyter Lab отсутствовал (был только классический notebook), поэтому установлен отдельно через pip с флагом `--break-system-packages` (рис. -Рисунок 4.3).

```

umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ pip install jupyterlab --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting jupyterlab
  Downloading jupyterlab-4.6.3-py3-none-any.whl.metadata (16 kB)
Collecting async-lru>=1.0.0 (from jupyterlab)
  Downloading async_lru-2.3.0-py3-none-any.whl.metadata (7.6 kB)
Collecting httpx<1,>=0.25.0 (from jupyterlab)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: ipykernel!=6.30.0,>=6.5.0 in /usr/lib/python3/dist-packages (from jupyterlab) (6.29.3)
Requirement already satisfied: Jinja2>=3.0.3 in /usr/lib/python3/dist-packages (from jupyterlab) (3.1.2)
Collecting jupyter-builder>=1.0.2 (from jupyterlab)
  Downloading jupyter_builder-1.2.3-py3-none-any.whl.metadata (7.8 kB)
Requirement already satisfied: jupyter-core in /usr/lib/python3/dist-packages (from jupyterlab) (5.3.2)
Collecting jupyter-lsp>=2.0.0 (from jupyterlab)
  Downloading jupyter_lsp-2.3.1-py3-none-any.whl.metadata (1.8 kB)
Collecting jupyter-server<3,>=2.19.0 (from jupyterlab)
  Downloading jupyter_server-2.21.0-py3-none-any.whl.metadata (8.5 kB)
Collecting jupyterlab-server<3,>=2.28.0 (from jupyterlab)
  Downloading jupyterlab_server-2.28.0-py3-none-any.whl.metadata (5.9 kB)
Collecting notebook>=6.2 (from jupyterlab)

```

Рисунок 4.3: Установка Jupyter Lab

Создан каталог для проекта лабораторной внутри репозитория курса (рис. -Рисунок 4.4).

```

umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ mkdir -p labs/lab01/project

```

Рисунок 4.4: Создание каталога labs/lab01/project

После добавления `~/local/bin` в PATH Jupyter Lab успешно запущен (рис.

-Рисунок 4.5).

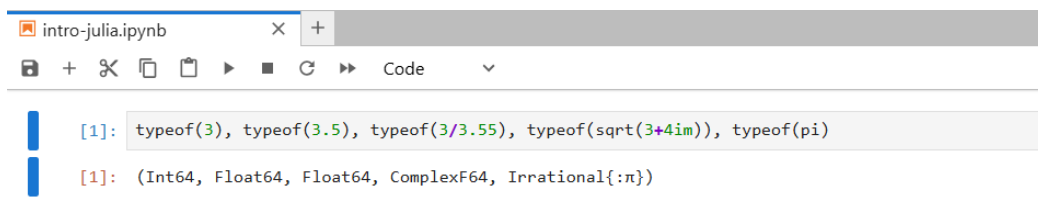
```
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ jupyter l
ab --version
4.6.3
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ jupyter l
ab
[I 2026-09-12 16:42:22.967 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-09-12 16:42:22.971 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-09-12 16:42:22.976 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-09-12 16:42:22.978 ServerApp] Writing Jupyter server cookie secret to /home/umedn/.local/share
/jupyter/runtime/jupyter_cookie_secret
[I 2026-09-12 16:42:23.387 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-09-12 16:42:23.450 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-09-12 16:42:23.452 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-09-12 16:42:23.454 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-09-12 16:42:23.457 LabApp] JupyterLab extension loaded from /home/umedn/.local/lib/python3.12/
site-packages/jupyterlab
[I 2026-09-12 16:42:23.458 LabApp] JupyterLab application directory is /home/umedn/.local/share/jupyter
lab
[I 2026-09-12 16:42:23.458 LabApp] Extension Manager is 'pypi'.
[I 2026-09-12 16:42:23.510 ServerApp] jupyterlab | extension was successfully loaded.
[I 2026-09-12 16:42:23.511 ServerApp] Serving notebooks from local directory: /home/umedn/work/study/2
026-2/2026-2==computer-practice/2026-2--study--computer-practice
```

Рисунок 4.5: Запуск Jupyter Lab

4.2 Основы синтаксиса Julia (примеры раздела 1.3.3)

В блокноте `intro-julia.ipynb` с ядром Julia 1.10 последовательно выполнены примеры из методических указаний.

Определение типа числовой величины для целого, вещественного, комплексного числа и результата операции даёт ожидаемые типы `Int64`, `Float64`, `Complex{Float64}`, `Irrational` (рис. -Рисунок 4.6).



```
intro-julia.ipynb
[1]: typeof(3), typeof(3.5), typeof(3/3.55), typeof(sqrt(3+4im)), typeof(pi)
[1]: (Int64, Float64, Float64, ComplexF64, Irrational{::pi})
```

Рисунок 4.6: Определение типа числовых величин

Деление на ноль для вещественных чисел даёт специальные значения `Inf`, `-Inf`, `NaN` (рис. -Рисунок 4.7), которые имеют тип `Float64` (рис. -Рисунок 4.8).

```
[2]: 1.0/0.0, 1.0/(-0.0), 0.0/0.0
```

```
[2]: (Inf, -Inf, NaN)
```

Рисунок 4.7: Специальные значения Inf, -Inf, NaN

```
[3]: typeof(1.0/0.0), typeof(1.0/-0.0), typeof(0.0/0.0)
```

```
[3]: (Float64, Float64, Float64)
```

Рисунок 4.8: Тип специальных значений

Цикл по всем целочисленным типам выводит границы их диапазонов (рис. - Рисунок 4.9).

```
[6]: for T in [Int8, Int16, Int32, Int64, Int128, UInt8, UInt16, UInt32, UInt64, UInt128]
      println("${lpad(T,7)}: [$(typemin(T)),$(typemax(T))]" )
    end

Int8: [-128,127]
Int16: [-32768,32767]
Int32: [-2147483648,2147483647]
Int64: [-9223372036854775808,9223372036854775807]
Int128: [-170141183460469231731687303715884105728,170141183460469231731687303715884105727]
UInt8: [0,255]
UInt16: [0,65535]
UInt32: [0,4294967295]
UInt64: [0,18446744073709551615]
UInt128: [0,340282366920938463463374607431768211455]
```

Рисунок 4.9: Диапазоны целочисленных типов

Прямое преобразование типов конструкторами Int64() и Char() (рис. -Рисунок 4.10) и эквивалентное преобразование через convert() (рис. - Рисунок 4.11) дают одинаковый результат.

```
[7]: Int64(2.0), Char(2)
```

```
[7]: (2, '\x02')
```

Рисунок 4.10: Преобразование типов конструкторами

```
[8]: convert(Int64, 2.0), convert(Char, 2)
```

```
[8]: (2, '\x02')
```

Рисунок 4.11: Преобразование типов через `convert()`

Преобразование числовых значений 1 и 0 в булев тип (рис. -Рисунок 4.12).

```
[9]: Bool(1), Bool(0)
```

```
[9]: (true, false)
```

Рисунок 4.12: Преобразование в `Bool`

Функция `promote()` приводит аргументы разных числовых типов (`Int8`, `Float16`, `Float32`) к общему типу `Float32` (рис. -Рисунок 4.13), что подтверждается функцией `typeof()` (рис. -Рисунок 4.14).

```
[10]: promote(Int8(1), Float16(4.5), Float32(4.1))
```

```
[10]: (1.0f0, 4.5f0, 4.1f0)
```

Рисунок 4.13: Приведение аргументов к общему типу

```
[11]: typeof(promote(Int8(1), Float16(4.5), Float32(4.1)))
```

```
[11]: Tuple{Float32, Float32, Float32}
```

Рисунок 4.14: Тип результата `promote()`

Определение функции полной формой (`function ... end`) на примере

возведения в квадрат (рис. -Рисунок 4.15) и краткой однострочной формой (рис. -Рисунок 4.16) дают одинаковый результат.

```
[12]: function f(x)
      x^2
end
f(4)
```

```
[12]: 16
```

Рисунок 4.15: Функция в полной форме

```
[13]: g(x) = x^2
      g(8)
```

```
[13]: 64
```

Рисунок 4.16: Функция в краткой форме

Определение одномерных массивов — вектора-строки и вектора-столбца — и обращение к их элементам по индексу (рис. -Рисунок 4.17).

```
[14]: a = [4 7 6]
      b = [1, 2, 3]
      a[2], b[2]
```

```
[14]: (7, 2)
```

Рисунок 4.17: Векторы и обращение к элементам

Определение двумерного массива (матрицы) и обращение к её элементам по паре индексов (рис. -Рисунок 4.18).

```
[15]: a = 1; b = 2; c = 3; d = 4
      Am = [a b; c d]
      Am[1,1], Am[1,2], Am[2,1], Am[2,2]
```

```
[15]: (1, 2, 3, 4)
```

Рисунок 4.18: Матрица и обращение к элементам

Умножение вектора-строки на матрицу и на транспонированный вектор-столбец (рис. -Рисунок 4.19).

```
[16]: aa = [1 2]
      AA = [1 2; 3 4]
      aa*AA*aa'
```

```
[16]: 1x1 Matrix{Int64}:
      27
```

Рисунок 4.19: Операция с транспонированием вектора

4.3 Задания для самостоятельной работы (раздел 1.3.4)

4.3.1 Задание 1. Функции чтения/записи/вывода информации

Сравнение `print()`, `println()` и `show()` показывает разницу в оформлении вывода: `print/println` выводят строку как есть, а `show` — в «программном» виде (строка в кавычках), что видно на примере с текстом «текст с кавычками» (рис. -Рисунок 4.20).

```
[17]: print("Hello"); print(", world!\n")
      println("Hello, world!")
      show(1//3); println()
      show("текст с кавычками"); println()
      print("текст с кавычками")

Hello, world!
Hello, world!
1//3
"текст с кавычками"
текст с кавычками
```

Рисунок 4.20: `print`, `println` и `show`

Функция `write()` записывает строку как байты в поток — в примере в буфер `IOBuffer`, при этом возвращает число записанных байт (рис. -Рисунок 4.21).

```
[18]: io = IOBuffer()  
      n = write(io, "запись в буфер\n")  
      println("записано байт: ", n)  
      println(String(take!(io)))
```

```
записано байт: 27  
запись в буфер
```

Рисунок 4.21: write() в буфер

Файл записан построчно через `write()`, после чего продемонстрирована разница между `read()` (весь файл одной строкой), `readline()` (только первая строка потока) и `readlines()` (вектор всех строк) (рис. -Рисунок 4.22).

```
[19]: open("demo.txt", "w") do f
      write(f, "строка 1\n")
      write(f, "строка 2\n")
      write(f, "строка 3\n")
    end

    println("read() -- весь файл одной строкой:")
    println(read("demo.txt", String))

    println("readline() -- только первая строка:")
    open("demo.txt") do f
      println(readline(f))
    end

    println("readlines() -- вектор всех строк:")
    println(readlines("demo.txt"))

    read() -- весь файл одной строкой:
    строка 1
    строка 2
    строка 3

    readline() -- только первая строка:
    строка 1
    readlines() -- вектор всех строк:
    ["строка 1", "строка 2", "строка 3"]
```

[]:

Рисунок 4.22: read(), readline(), readlines()

Функция `readdlm()` из стандартного пакета `DelimitedFiles` разбирает файл с разделителем-запятой в числовую матрицу (рис. -Рисунок 4.23).

```
[20]: using DelimitedFiles

open("table.csv", "w") do f
    write(f, "1,2,3\n4,5,6\n")
end

data = readdlm("table.csv", ',', Int)
println(data)

rm("demo.txt"); rm("table.csv")

[1 2 3; 4 5 6]
```

Рисунок 4.23: readdlm() для табличных данных

4.3.2 Задание 2. Функция parse()

Базовое использование parse() для получения Int, Float64 и Bool из строк (рис. -Рисунок 4.24).

```
[22]: parse(Int, "42"), parse(Float64, "3.14"), parse(Bool, "true")

[22]: (42, 3.14, true)
```

Рисунок 4.24: parse() для базовых типов

Именованный аргумент base позволяет разбирать строку в заданной системе счисления — двоичной и шестнадцатеричной (рис. -Рисунок 4.25).

```
[23]: parse(Int, "101", base=2), parse(Int, "ff", base=16)

[23]: (5, 255)
```

Рисунок 4.25: parse() с указанием системы счисления

`tryparse()` – безопасный аналог `parse()`: при невозможности разбора возвращает `nothing` вместо исключения (рис. -Рисунок 4.26).

```
[24]: tryparse(Int, "42"), tryparse(Int, "не число")
```

```
[24]: (42, nothing)
```

Рисунок 4.26: `tryparse()`

4.3.3 Задание 3. Базовые математические операции с разными типами

При смешении `Int` и `Float64` результат автоматически приводится к `Float64`, а `Rational` (`5//2`) сохраняет свой тип, пока в выражении не появится вещественное число (рис. -Рисунок 4.27).

```
[25]: 5 + 3, 5 + 3.0, 5 // 2, 5 // 2 + 0.5
```

```
[25]: (8, 8.0, 5//2, 3.0)
```

Рисунок 4.27: Арифметика со смешанными типами

Оператор `/` всегда возвращает `Float64`, `÷` выполняет целочисленное деление, `%` – вычисляет остаток (рис. -Рисунок 4.28).

```
[26]: 7 / 2, 7 ÷ 2, 7 % 2
```

```
[26]: (3.5, 3, 1)
```

Рисунок 4.28: Операторы деления и остатка

Возведение в степень (^) и извлечение корней (sqrt, cbrt) на примере степени 0.5 как альтернативы sqrt() (рис. -Рисунок 4.29).

```
[27]: 2^10, sqrt(16), 16^0.5, cbrt(27)

[27]: (1024, 4.0, 4.0, 3.0)
```

Рисунок 4.29: Степени и корни

Оператор == сравнивает значения независимо от типа, === — дополнительно сравнивает тип, а сравнения можно объединять в цепочку (рис. -Рисунок 4.30).

```
[33]: 3 == 3.0, 3 === 3.0, 1 < 2 < 3

[33]: (true, false, true)
```

Рисунок 4.30: Сравнение значений и типов

Логические операции &&, ||, !, xor() (рис. -Рисунок 4.31).

```
[34]: true && false, true || false, !true, xor(true, false)

[34]: (false, true, false, true)
```

Рисунок 4.31: Логические операции

Арифметика с комплексными числами: сложение, умножение и вычисление модуля (рис. -Рисунок 4.32).

```
[35]: (1+2im) + (3-1im), (1+2im) * (3-1im), abs(3+4im)
```

```
[35]: (4 + 1im, 5 + 5im, 5.0)
```

Рисунок 4.32: Операции с комплексными числами

4.3.4 Задание 4. Операции над матрицами и векторами

Поэлементное сложение, вычитание векторов и умножение вектора на скаляр (рис. -Рисунок 4.33).

```
[36]: v1 = [1, 2, 3]
```

```
v2 = [4, 5, 6]
```

```
v1 + v2, v1 - v2, 3 * v1
```

```
[36]: ([5, 7, 9], [-3, -3, -3], [3, 6, 9])
```

Рисунок 4.33: Операции над векторами

Скалярное произведение векторов вычислено двумя способами: функцией `dot()` из пакета `LinearAlgebra` и через умножение транспонированного вектора на вектор (рис. -Рисунок 4.34).

```
[37]: using LinearAlgebra  
  
dot(v1, v2), v1' * v2
```

```
[37]: (32, 32)
```

Рисунок 4.34: Скалярное произведение векторов

Сложение и вычитание матриц выполняются поэлементно (рис. -Рисунок 4.35).

```
[38]: A = [1 2; 3 4]  
      B = [5 6; 7 8]  
  
      A + B, A - B
```

```
[38]: ([6 8; 10 12], [-4 -4; -4 -4])
```

Рисунок 4.35: Сложение и вычитание матриц

Транспонирование матрицы через оператор ' и функцию `transpose()` даёт одинаковый результат для вещественной матрицы (рис. -Рисунок 4.36).

```
[39]: A', transpose(A)
```

```
[39]: ([1 3; 2 4], [1 3; 2 4])
```

Рисунок 4.36: Транспонирование матрицы

Умножение матрицы на скаляр, матричное умножение $A * B$ и поэлементное умножение $A .* B$ дают разные результаты (рис. -Рисунок 4.37).

```
[40]: 2 * A, A * B, A .* B
```

```
[40]: ([2 4; 6 8], [19 22; 43 50], [5 12; 21 32])
```

Рисунок 4.37: Умножение матриц: на скаляр, матричное и поэлементное

5 Результаты

- Julia 1.10.5, Jupyter Lab 4.6.3 и ядро IJulia установлены и настроены, блокнот `intro-julia.ipynb` создан и выполнен с ядром Julia.
- Все примеры раздела 1.3.3 воспроизведены, результаты совпадают с методическими указаниями.
- Задание 1 выполнено: показана разница между `print/println/show`, продемонстрированы `write()`, `read()`, `readline()`, `readlines()`, `readdlm()`.
- Задание 2 выполнено: показана работа `parse()` с базовыми типами, с указанием системы счисления и безопасный разбор через `tryparse()`.
- Задание 3 выполнено: рассмотрены арифметические операции со смешанными типами, деление и остаток, степени и корни, сравнение значений и типов, логические операции, арифметика комплексных чисел.
- Задание 4 выполнено: рассмотрены сложение/вычитание векторов и матриц, умножение на скаляр, скалярное произведение, транспонирование, матричное и поэлементное умножение.

6 Обсуждение результатов

Результаты выполнения примеров и заданий полностью совпали с ожидаемым поведением, описанным в теоретической части: система типов Julia действительно повышает тип при смешанных операциях, а специализированные операторы ($\div$, $//$, $\cdot$, $*$) сохраняют математический смысл, отличный от их «обычных» аналогов ($/$, $*$).

В процессе установки возникла техническая сложность: после установки Jupyter Lab через `pip install jupyterlab --break-system-packages` команда `jupyter lab` не находилась, так как каталог `~/ .local/bin` отсутствовал в переменной `PATH`. Проблема решена добавлением каталога в `PATH` через `~/ .bashrc`.

Также при наборе выражения `3 == 3.0, 3 === 3.0, 1 < 2 < 3` возникла ошибка `not a unary operator`, вызванная переносом строки прямо перед оператором `===` — Julia интерпретировала часть после переноса как новое выражение, начинающееся со знака `=`. После записи выражения одной строкой без переносов код выполнялся корректно.

7 Заключение

- Поставленная цель работы достигнута: рабочее окружение для Julia подготовлено, основы синтаксиса освоены на практике.
- Получены навыки установки и настройки Julia, Jupyter Lab и IJulia в среде WSL Ubuntu.
- Изучены и опробованы на собственных примерах функции ввода-вывода, `parse()/tryparse()`, арифметические и логические операторы, операции над матрицами и векторами.
- Рекомендация: при дальнейшей работе с Julia в Jupyter обращать внимание на перенос строк в многострочных выражениях REPL/ячеек, чтобы избежать ошибок разбора.

Список литературы

1. Julia 1.5 Documentation. — 2020. — URL: <https://docs.julialang.org/en/v1/>.
2. Klok H., Nazarathy Y. Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. — 2020. — URL: <https://statisticswithjulia.org/>.
3. Ökten G. First Semester in Numerical Analysis with Julia. — Florida State University, 2019.
4. Антонюк В. А. Язык Julia как инструмент исследователя. — М. : Физический факультет МГУ им. М. В. Ломоносова, 2019.
5. Шиндин А. В. Язык программирования математических вычислений Julia. Базовое руководство. — Нижний Новгород : Нижегородский госуниверситет, 2016.

А Листинг кода блокнота

intro-julia.ipynb

```
# --- 1.3.3. Julia ---

typeof(3), typeof(3.5), typeof(3/3.55), typeof(sqrt(3+4im)), typeof(
1.0/0.0, 1.0/(-0.0), 0.0/0.0

typeof(1.0/0.0), typeof(1.0/-0.0), typeof(0.0/0.0)

for T in [Int8, Int16, Int32, Int64, Int128, UInt8, UInt16, UInt32, UI
    println("$(@pad(T,7)): [$(typemin(T)),$(typemax(T))]" )
end

Int64(2.0), Char(2)

convert(Int64, 2.0), convert(Char, 2)

Bool(1), Bool(0)

promote(Int8(1), Float16(4.5), Float32(4.1))
```

```
typeof(promote(Int8(1), Float16(4.5), Float32(4.1)))
```

```
function f(x)
```

```
    x^2
```

```
end
```

```
f(4)
```

```
g(x) = x^2
```

```
g(8)
```

```
a = [4 7 6]
```

```
b = [1, 2, 3]
```

```
a[2], b[2]
```

```
a = 1; b = 2; c = 3; d = 4
```

```
Am = [a b; c d]
```

```
Am[1,1], Am[1,2], Am[2,1], Am[2,2]
```

```
aa = [1 2]
```

```
AA = [1 2; 3 4]
```

```
aa*AA*aa'
```

```
# --- 1.3.4. 1: / / ---
```

```
print("Hello"); print(", world!\n")
```

```
println("Hello, world!")
```

```
show(1//3); println()
```

```
show("  1  2  3  4  5  6  7  8  9  10"); println()
```

```

print("XXXXX X XXXXXXXX")

io = IOBuffer()
n = write(io, "XXXXX X XXXX\n")
println("XXXXXXX XXX: ", n)
println(String(take!(io)))

open("demo.txt", "w") do f
    write(f, "XXXXX 1\n")
    write(f, "XXXXX 2\n")
    write(f, "XXXXX 3\n")
end

println("read() -- XXXX XXXX XXXX XXXXXX:")
println(read("demo.txt", String))

println("readline() -- XXXXXX XXXXXX XXXXXX:")
open("demo.txt") do f
    println(readline(f))
end

println("readlines() -- XXXXXX XXXX XXXXX:")
println(readlines("demo.txt"))

using DelimitedFiles

open("table.csv", "w") do f
    write(f, "1,2,3\n4,5,6\n")
end

```

end

```
data = readdlm("table.csv", ',', Int)
```

```
println(data)
```

```
rm("demo.txt"); rm("table.csv")
```

```
# --- 2: parse() ---
```

```
parse(Int, "42"), parse(Float64, "3.14"), parse(Bool, "true")
```

```
parse(Int, "101", base=2), parse(Int, "ff", base=16)
```

```
tryparse(Int, "42"), tryparse(Int, "x xxxx")
```

```
# --- 3: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 ---
```

```
5 + 3, 5 + 3.0, 5 // 2, 5 // 2 + 0.5
```

```
7 / 2, 7 ÷ 2, 7 % 2
```

```
2^10, sqrt(16), 16^0.5, cbrt(27)
```

```
3 == 3.0, 3 === 3.0, 1 < 2 < 3
```

```
true && false, true || false, !true, xor(true, false)
```

```
(1+2im) + (3-1im), (1+2im) * (3-1im), abs(3+4im)
```

```
# --- 4:  x  ---
```

```
v1 = [1, 2, 3]
```

```
v2 = [4, 5, 6]
```

```
v1 + v2, v1 - v2, 3 * v1
```

```
using LinearAlgebra
```

```
dot(v1, v2), v1' * v2
```

```
A = [1 2; 3 4]
```

```
B = [5 6; 7 8]
```

```
A + B, A - B
```

```
A', transpose(A)
```

```
2 * A, A * B, A .* B
```